



Tarea 1: Fundamentos de RL (2026-S1)

Problema 1: Gestión óptima de energía en Microrred Solar (1 punto)

Debido a sus altos conocimientos en control inteligente, pero más específicamente en aprendizaje reforzado, Alan Brito lo ha contratado para que implemente un controlador que permita gestionar de manera eficiente la energía de su nueva isla recién comprada a su buen amigo Jeffrey.

Para abastecer la demanda de electricidad de la isla, se ha instalado una microrred local compuesta por: (i) Paneles solares, (ii) Una Batería y (iii) Un Generador Diesel de respaldo.

Su controlador deberá tomar decisiones inteligentes sobre cómo utilizar la batería a lo largo del día con el objetivo de operar el sistema de manera eficiente y sostenible. En particular, se desea:

- Reducir al máximo el uso del generador diésel, debido a su alto costo económico y ambiental.
- Evitar situaciones en las que no haya suficiente energía disponible para cubrir la demanda, especialmente de noche cuando no existe generación solar.
- Limitar el uso excesivo de la batería, ya que cargarla o descargarla implica desgaste y costos operacionales.

El controlador puede tomar decisiones únicamente en tres momentos del día: mañana (*m: morning*), tarde (*a: afternoon*) y noche (*n: night*). En cada uno de estos periodos dispone de cuatro acciones posibles:

- -2: Descargar la batería intensamente para inyectar energía al sistema.
- -1: Descarga la batería levemente para inyectar energía al sistema.
- 0: No utilizar la batería.
- 1: Cargar la batería extrayendo energía de los paneles.

Por otro lado, en cada instante y antes de tomar su decisión, el controlador podrá observar las siguientes variables:

- B_t : Nivel de batería, con seis niveles (0, 1, 2, 3, 4, 5), el cual es afectado directamente por las acciones de control. Es decir, si el nivel de batería es $B_t = 4$ y la acción es descargar la batería intensamente ($a_t = -2$), entonces $B_{t+1} = 2$
- D_t : Momento del día (*m, a, n*)
- L_t : Demanda de energía observada, que depende del momento del día y de la demanda anterior L_{t-1}
- G_t : Generación solar observada, que depende del momento del día y de la generación anterior G_{t-1}

Hint 1: Es importante recalcar que la acción tomada en el instante t afecta la evolución del sistema en el instante $t + 1$. En particular, el controlador debe decidir cómo utilizar la batería considerando que su decisión influirá en la capacidad del sistema para cubrir la demanda futura.

Hint 2: En el script asociado a esta pregunta, llamado `p1.py` se encuentran las probabilidades de transición para L_t y G_t , es decir $p(L_{t+1}|L_t, D_t)$ y $p(G_{t+1}|G_t, D_t)$.

Su Tarea

- Defina formalmente el espacio de estados $\mathcal{S}$ y el espacio de acciones $\mathcal{A}$. Indique claramente qué variables componen cada estado y determine la dimensión total del espacio de estados.
- Defina una función de recompensa que cumpla con todos los requisitos solicitados.
- Dado lo definido en a) y b), escriba explícitamente todos los posibles estados siguientes s_{t+1} y sus respectivas probabilidades de transición para $s_t = (B_t = 2, D_t = m, L_t = 3, G_t = 2)$ y $a_t = -1$. Luego, escriba la ecuación de Bellman tabular ($v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s'|s,a)[r + \gamma v_\pi(s')]$), considerando un controlador determinístico y que todos los valores fueron inicializados en cero.
- Implemente los algoritmos *Policy Iteration* y *Value Iteration*, y utilícelos para resolver el problema. Reporte el número de iteraciones necesarias hasta la convergencia en cada caso. Compare ambos métodos y comente brevemente las diferencias observadas.
- Reporte la política óptima obtenida e interprétela cualitativamente. Comente, al menos:
 - En qué situaciones conviene cargar la batería,
 - En qué situaciones conviene descargarla,
 - Cómo influyen el momento del día, la demanda observada y la generación solar observada en la decisión óptima.
 - Seleccione valores para tres días seguidos de Generación y Demanda y analice la respuesta de su controlador.

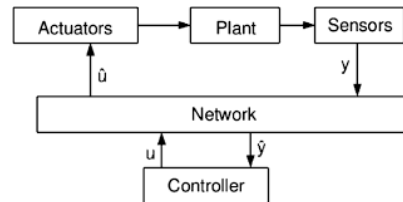
Problema 2: Networked Control Systems (2 puntos)

Usted ha sido el afortunado seleccionado para modernizar la famosa experiencia del *Twin Rotor* del laboratorio de control. Este dispositivo, mostrado en la Figura 1a), está descrito por las siguientes ecuaciones de movimiento:

$$\begin{aligned}
 \ddot{\phi} &= \frac{1}{J_p} (K_{pp} V_m + K_{pt} V_t - mgl \cos(\phi) \cos(\psi) - B_p \dot{\phi} + J_r \Omega \dot{\psi}) \quad (\text{pitch}) \\
 \ddot{\psi} &= \frac{1}{J_y} (K_{yp} V_m + K_{yt} V_t - B_y \dot{\psi} - J_r \Omega \dot{\phi}) \quad (\text{yaw}) \\
 \dot{V}_m &= \frac{1}{T_m} (-V_m + V_{m,cmd}) \quad (\text{actuador principal}) \\
 \dot{V}_t &= \frac{1}{T_t} (-V_t + V_{t,cmd}) \quad (\text{actuador de la cola})
 \end{aligned} \tag{1}$$



(a) Twin Rotor



(b) Networked Control System

Figura 1: Diagrama del Twin Rotor y un Networked Control System

En el marco de esta modernización, se desea permitir que los estudiantes puedan operar el sistema de forma remota a través de una red de comunicaciones, como se ilustra en la Figura 1b).

Sin embargo, la transmisión de comandos de control no es perfecta. El laboratorio dispone de distintos canales de comunicación inalámbrica, cada uno con una probabilidad de transmisión exitosa $\theta \in [0, 1]$. En cada instante de tiempo, un comando puede perderse debido a interferencias, congestión de red o limitaciones de hardware. Cuando esto ocurre, el actuador mantiene el último comando recibido correctamente, lo que puede deteriorar significativamente el desempeño del controlador o incluso comprometer la estabilidad del sistema. Por lo que usted debe encontrar la forma de que cada vez que el sistema se conecte, encuentre de manera automática el mejor canal de comunicación disponible.

Para probar sus ideas se le ha entregado los siguientes scripts:

- **TwinRotorODE.py**: Contiene una clase para simular la dinámica no lineal y linealizada del sistema *Twin Rotor*. En particular, la función `xd_func` implementa las ecuaciones del sistema en espacio de estados.
- **p3.py**: Script de ejemplo que muestra cómo simular el sistema no lineal, cómo obtener las matrices discretas del modelo linealizado para una referencia dada y cómo utilizar el Filtro de Kalman adjunto para estimar el estado del sistema a partir de mediciones ruidosas.
- **KF.py**: Una implementación del Filtro de Kalman discreto (no debe ser modificado).
- **channel_selector.py**: Contiene la clase *ChannelSelector*, la cual permite simular fallas en los distintos canales de comunicación. Un ejemplo de uso se encuentra al final del script. En cada instante de tiempo, el canal puede funcionar o fallar. Si el canal funciona, la variable booleana `success` toma el valor **True**; en caso contrario, toma el valor **False**. Considere que, cuando ocurre una falla en el canal, el sistema utiliza la última medición o la última acción de control disponible correspondiente al último instante en que la comunicación fue exitosa.

Su Tarea

- a) Ejecute el archivo **p3.py** y comente las razones de las diferencias observadas entre la respuesta del sistema no lineal, el modelo linealizado y la estimación obtenida mediante el Filtro de Kalman. Investigue brevemente el principio de funcionamiento del Filtro de Kalman y su rol en sistemas de control.
- b) Simule el sistema no lineal en lazo abierto, inicializado en el punto de equilibrio correspondiente a $x_0 = 0$ y $x_1 = 0$. Aplique un escalón en la entrada asociada al *pitch* (primera entrada) y luego en la entrada asociada al *yaw* (segunda entrada). Grafique las respuestas obtenidas. A partir de los resultados, responda: ¿por qué un controlador PID independiente para cada eje no sería efectivo en este sistema?
- c) Utilizando programación dinámica, obtenga la ganancia K del controlador LQR considerando un horizonte finito suficientemente largo hasta que la matriz de Riccati converja. Utilice la ganancia obtenida en el punto de convergencia y compárela con la matriz K entregada por la librería **control** de Python mediante el comando:

$$K, P, E = \text{control.dlqr}(Ad, Bd, Q, R).$$

- d) Cierre el lazo de control para regular el sistema no lineal. Simule el sistema durante 2000 pasos y verifique que el controlador es capaz de rechazar perturbaciones externas.

Considere que el estado completo del sistema no es directamente medible, por lo que deberá estimarlo utilizando el Filtro de Kalman y emplear esta estimación para calcular la acción de control. Posteriormente, modifique la referencia a $x_0 = 0, x_1 = \frac{\pi}{4}$ y luego a $x_0 = \frac{\pi}{4}, x_1 = \frac{\pi}{4}$ (lo que implica volver a linearizar el sistema y recalculer las matrices correspondientes). Evalúe nuevamente la capacidad de rechazo de perturbaciones. Grafique sus resultados y utilice una métrica cuantitativa de desempeño, como el *Integral Absolute Error* (IAE), para comparar los distintos escenarios. Las perturbaciones pueden añadirse como sigue:

```

1 system = TwinRotorEnhanced(params, Ts, disturbance_time=[1000, 1500], disturbance_strength
    =0.1)

```

Código 1: Ejemplo de simulación con perturbaciones

- e) Finalmente, instancie la clase *ChannelSelector* para simular la comunicación inalámbrica entre la planta y el controlador, considerando que tanto las mediciones como las acciones de control se transmiten a través de canales imperfectos. Suponga que en cada paso de tiempo solo es posible seleccionar un único canal de comunicación y que debido a la latencia del sistema deberá utilizar un $T_s = 0.05$. Utilice el algoritmo *Thompson Sampling* para identificar el canal más confiable a lo largo del tiempo. Grafique el reward acumulado, considerando que **Success=True** implica $r = 1$ y **Success=False** implica $r = 0$.

Además, grafique las distribuciones Beta aprendidas para cada canal (puede utilizar `beta.pdf(x, alpha[i], beta_param[i])` de la librería `scipy.stats`). Finalmente, compare este enfoque con una estrategia ϵ -greedy en términos de *regret* acumulado y discuta sus resultados.

Problema 3: Diseño de salidas de emergencia con simulación Multi-Agente (3 puntos)

Dentro de la isla, Alan Brito quiere construir un edificio que será la sede de su nueva empresa de inteligencia artificial, *Open Brain*. Sin embargo, hay un pequeño problema: la isla es propensa a terremotos e incendios. A pesar de ello, Alan Brito no está dispuesto a gastar demasiado dinero, por lo que le pidió al arquitecto que diseñara un edificio *low cost*. El plano de cada piso (todos exactamente iguales) se muestra en la figura 2.

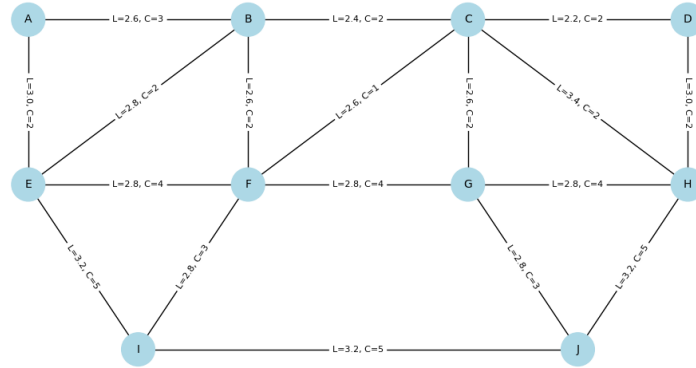


Figura 2: Plano de cada piso del edificio de *Open Brain*

Cada nodo representa una oficina, las cuales tienen una capacidad máxima de 5 personas. Por su parte, cada arista representa un pasillo que conecta distintas oficinas. Estos pasillos tienen un largo L (en metros) y una capacidad máxima aproximada C de personas que pueden transitar simultáneamente.

Dado este plano, usted ha sido encomendado con la tarea de determinar la mejor ubicación para instalar dos salidas de emergencia, S_1 y S_2 , con el objetivo de maximizar la cantidad de personas evacuadas en caso de un sismo o incendio. En particular, se busca que la mayor cantidad posible de personas alcance alguna de las salidas dentro de un periodo de 5 minutos (300 segundos).

Para abordar este problema, usted contará con un simulador adjunto a esta tarea en el archivo `evacuation.py`, el cual funciona de la siguiente manera:

- El estado se define como una tupla de 6 valores, por ejemplo $(A, 3, 2, -1, -1, -1)$. El primer valor indica el nodo en el cual se encuentra el agente. Los valores siguientes distintos de -1 representan la cantidad de personas

transitando por los pasillos que conectan el nodo actual con sus nodos vecinos, ordenados alfabéticamente según el nombre del vecino. En nuestro ejemplo, el segundo valor indica que en el pasillo $A-B$ hay 3 personas transitando, mientras que el tercer valor indica que en el pasillo $A-E$ hay 2 personas. Los valores iguales a -1 corresponden a valores *dummy* que se utilizan únicamente para que todos los estados tengan la misma dimensión.

- El espacio de acciones en cada estado depende de la cantidad de vecinos del nodo actual. Cada acción corresponde a elegir uno de los pasillos disponibles, siguiendo nuevamente el orden alfabético de los vecinos. En el ejemplo anterior, la acción $a = 0$ indica que el agente decide transitar por el pasillo $A-B$, mientras que la acción $a = 1$ indica que el agente transita por el pasillo $A-E$. Cualquier valor de acción mayor o igual al número de vecinos disponibles (o $a = -1$) será ignorado, y se interpretará como que el agente permanece en el nodo actual.
- En cada paso de tiempo, el sistema funciona como sigue:
 1. El agente observa su estado actual (es decir, el nodo en el que se encuentra y la ocupación de los pasillos adyacentes) y ejecuta una acción.
 2. Todos los demás agentes realizan simultáneamente sus acciones, tras lo cual se actualizan las ocupaciones de los pasillos correspondientes.
 3. Utilizando la ocupación actualizada del pasillo seleccionado, junto con su capacidad y largo, se calcula un tiempo estimado de permanencia en el pasillo τ . El agente recibe un *reward* inmediato igual a $r = -\tau$, lo cual incentiva el uso de pasillos más cortos y menos congestionados. Si el agente decide transitar por un pasillo que lo conduce a una salida de emergencia, el *reward* recibido es $r = -\tau + 1000$.
 4. Una vez que el agente entra a un pasillo, no puede ejecutar nuevas acciones hasta que el tiempo de permanencia τ se cumpla completamente.
 5. Solo cuando el agente llega físicamente al siguiente nodo, puede volver seleccionar una acción. En el caso de alcanzar una salida de emergencia, su recorrido se considera finalizado.
- La interacción con el simulador se realiza mediante el método `step`, como se muestra en el Código 2 (un ejemplo completo se encuentra en el archivo adjunto `p2.py`):

```

1 actions = [0] * env.n_agents
2 next_states, rewards, arrived_to_node, times_to_arrival, global_time, done_list,
  just_arrived_to_shelter, arrival_event_time = env.step(actions)

```

Código 2: Ejemplo de interacción con el simulador

Los valores retornados tienen el siguiente significado:

- **next_states**: Lista con los estados actuales de todos los agentes luego de aplicar las acciones. Si un agente se encuentra transitando por un pasillo, las ocupaciones de los pasillos sí se actualizan, pero el nodo del agente no cambia hasta que este llega físicamente al nodo de destino.
- **rewards**: Lista con los *rewards* obtenidos por cada agente. El *reward* solo se entrega cuando el agente ejecuta una acción al llegar a un nodo. Si el agente se encuentra en transición, el valor correspondiente es *None*.
- **arrived_to_node**: Lista de valores booleanos que indican si cada agente ha llegado a un nodo al final de la iteración actual.
- **times_to_arrival**: Lista con la cuenta regresiva (en pasos de simulación) para que cada agente llegue al siguiente nodo.
- **global_time**: Tiempo total de simulación (en segundos) transcurrido en el episodio actual. Cada paso del simulador corresponde a 5 segundos.
- **done_list**: Lista de valores booleanos que indican si cada agente ya ha alcanzado una salida de emergencia.

- `just_arrived_to_shelter`: Lista de valores booleanos que indica si cada agente alcanzó una salida de emergencia en esta iteración.
- `arrival_event_time`: Lista que contiene el tiempo de llegada de cada agente a una salida de emergencia. Si el agente aún no ha llegado, el valor correspondiente es *None*.

Hint 1: Cada paso del simulador representa 5 segundos, por ende, cada episodio debe durar 60 pasos ($60 \times 5 = 300$ segundos)

Su Tarea

- Investigue y explique **brevemente** qué es *Multi-Agent Reinforcement Learning* (MARL) y cuáles son sus principales desafíos. En particular, investigue qué se entiende por enfoques *centralized*, *decentralized* e *independent* en MARL. Finalmente, comente por qué, en aplicaciones como la de este problema, puede resultar más adecuado modelar un sistema como un problema de MARL en lugar de utilizar un único agente.
- Considerando las salidas de emergencia en las posiciones entregadas en el script `p2.py`, implemente el algoritmo de *Independent Multi-Agent SARSA* e *Independent Multi-Agent Q-learning* con 50 agentes, distribuidos inicialmente con 5 agentes por nodo. Explique brevemente los algoritmos implementados. Además, muestre el progreso del *reward* acumulado y la cantidad de personas salvadas en función de las épocas de entrenamiento en cada caso. Compare el desempeño de ambos algoritmos, seleccione el mejor y proponga una explicación que justifique dicha diferencia de rendimiento.
- Una vez finalizado el entrenamiento, simule nuevamente el sistema y muestre la cantidad de personas salvadas en función del tiempo, hasta un máximo de 300 segundos. Además, seleccione algunos agentes de distintos nodos e interprete sus trayectorias.
- Finalmente, evalúe distintas ubicaciones posibles para las salidas de emergencia y determine cuál corresponde a la mejor configuración. Para esta configuración muestre la cantidad de agentes que logran evacuar en función del tiempo. Compare los resultados obtenidos con la configuración original y discuta qué características de la topología del grafo o de la congestión explican el desempeño observado.

Anexo

Informe

El informe debe ser realizado en Latex, utilizando el template que está en Canvas.

Código

- Todo su código debe ser realizado en python version ≥ 3.11 .
- Está estrictamente prohibido el uso de Chatbots para completar su tarea. Si el profesor o los ayudantes determinan que utilizó un chatbot durante el desarrollo la tarea se calificará con Nota 1 (Ver diapositivas de la primera clase para el detalle de los lineamientos del curso en este tema).